

BestSupport Project Workflow Documentation

Scope: Django application workflow and code responsibilities. Deployment, Docker, and Kubernetes files are intentionally ignored.

1. Project Overview

BestSupport is a Django helpdesk system. Users create tickets, support users manage queues, ticket participants communicate through chat and calls, and special workflows exist for remote access approvals, CBS access requests, incident reports, technical documents, and portal announcements.

The important Django application folders are:

Path | Purpose

helpdesk/helpdesk/ | Django project package: global settings, root URL configuration, ASGI/WSGI entry points, and health endpoints.

helpdesk/accounts/ | User accounts, authentication views, custom user model, branches, departments, authentication settings, and password history.

helpdesk/tickets/ | Main helpdesk application: ticket models, forms, views, real-time consumers, notifications, storage helpers, and management commands.

helpdesk/templates/ | Shared and app-specific HTML templates used by the views.

helpdesk/static/ | Application CSS, JavaScript, and image assets.

2. Request Flow

- The browser sends an HTTP request to Django.
- helpdesk/helpdesk/urls.py decides whether the request belongs to admin, accounts, tickets, health, or the root redirect.
- For account pages, Django includes accounts.urls. For ticket pages, Django includes tickets.urls.
- The matched route calls a view function or class-based auth view.
- The view checks login and role permissions, loads models, validates forms, saves data, sends notifications or emails, and renders a template or redirects.
- Templates show the response using context data prepared by the view.
- For chat, call signaling, and browser notifications, ASGI routes WebSocket traffic to consumers in tickets/consumers.py.

3. Root URL Configuration

helpdesk/helpdesk/urls.py is the main URL entry point.

URL | Code | What It Does

/admin/ | admin.site.urls | Django admin interface.

/accounts/ | include("accounts.urls") | Authentication, signup, logout, dashboard, and password reset pages.

/tickets/ | include("tickets.urls") | Main helpdesk workflows.

/healthz/ | healthz | Basic health check endpoint.

/readyz/ | readyz | Readiness check endpoint.

/ | RedirectView | Redirects the root page to the login route.

4. Accounts App

Main Files

File | Responsibility

accounts/models.py | Defines CustomUser, Branch, Department, EmailSettings, AuthenticationSettings, and PasswordHistory.

accounts/forms.py | Signup and complete-signup form validation.

accounts/views.py | Signup, email verification, login, logout, password reset, and user dashboard.

accounts/urls.py | Maps account URLs to account views.

accounts/backends.py | Authentication backend logic, including local or directory login behavior.

accounts/middleware.py | Tracks active user sessions.

accounts/auth_mode.py | Central helper functions for authentication feature flags.

Account URLs and Views

URL | View | Purpose

/accounts/signup/ | signup | Starts local account registration by emailing a signed completion link.

/accounts/complete-signup/<token>/ | complete_signup | Validates the signed token and creates the user account.

/accounts/verify-email/<uidb64>/<token>/ | verify_email | Activates and verifies a user email when token validation succeeds.

/accounts/login/ | login_view | Authenticates username/email and password, logs the user in, and redirects to ticket list.

/accounts/logout/ | logout_view | Logs the user out and clears active sessions.

/accounts/dashboard/ | dashboard | Shows recent tickets and ticket counts for the logged-in user.

/accounts/password-reset/ | PasswordResetView | Sends password reset email when local self-service is enabled.

/accounts/reset/<uidb64>/<token>/ | PasswordResetConfirmView | Lets a user set a new password after token validation.

5. Tickets App: Main Concepts

Model | Purpose

Ticket | Main helpdesk record. Stores subject, request type, department, branch, priority, status, requester, assignee, notification emails, resolution/closure data, images, and metadata.

RemoteAccessApproval | Approval record linked to a ticket for remote access and CBS access workflows. Tracks recommender, second recommender, approver, decision status, signatures, and snapshots.

IncidentReport | Structured incident response report linked to an incident ticket, including signers, incident details, signatures, submission state, and correction data.

IncidentReportSignoff | Additional notified-user signoff rows for incident reports.

TicketMessage | Chat message inside a ticket conversation.

TicketMessageAttachment | Files attached to chat messages.

TicketAssignmentLog | History of assignment and status ownership changes.

TechnicalDocument | Uploaded PDF or Excel document with branch/department visibility rules.

PortalFlashAnnouncement | Portal announcement with schedule and image.

GroupMailboxEmail | Maps shared mailbox addresses to departments for ticket routing and notification.

Main Ticket Forms

Form | Used By | Purpose

TicketForm | create_ticket | Normal ticket creation, including department, branch, notify email, assignment email, impact, urgency, CC, and attachments.

RemoteAccessRequestForm | remote_access_request | Collects remote access request details and reviewer users.

CBSAccessRequestForm | cbs_access_request, cbs_access_branch_request | Collects CBS access request data and signoff chain users.

RemoteAccessApprovalDecisionForm | remote_access_approval_update | Records recommendation or approval decisions.

TicketUpdateForm | ticket_update | Support-user ticket updates including assignment, status, notes, resolved email attachments, and CC.

TicketAssigneeUpdateForm | ticket_update | Limited update form for the assigned agent.

IncidentReportForm | ticket_incident_report | Creates and updates structured incident reports.

TicketChatPrivacyForm | ticket_chat_privacy_update | Controls who can access ticket chat.

6. Ticket URLs and Views

URL | View | Purpose

/tickets/ | ticket_list | Shows tickets visible to the current user with filters and display flags.

/tickets/new/ | create_ticket | Creates a normal ticket, stores attachments, sends notifications, and creates an incident report template for incident tickets.

/tickets/<id>/ | ticket_detail | Shows full ticket information, messages, attachments, approval/report status, chat/call configuration, and allowed actions.

/tickets/<id>/claim/ | ticket_claim | Allows a department/support user to claim an unassigned ticket.

/tickets/support/<id>/update/ | ticket_update | Updates assignment, status, resolution, closure, notes, and requester emails.

/tickets/<id>/close/<token>/ | ticket_close_via_email | Lets the requester close a resolved ticket from a signed email link.

/tickets/<id>/attachments/upload/ | ticket_attachment_upload | Uploads chat attachments and broadcasts them to the ticket chat group.

/tickets/<id>/messages/<message_id>/delete/ | ticket_chat_message_delete | Deletes a chat message when the actor is allowed to delete it.

/tickets/<id>/chat/privacy/ | ticket_chat_privacy_update | Updates chat visibility rules for the ticket.

/tickets/<id>/chat/seen/ | ticket_chat_mark_seen | Marks chat messages as seen for the current user.

/tickets/<id>/image/view/ | ticket_image_view | Streams the ticket image for inline viewing.

/tickets/<id>/image/download/ | ticket_image_download | Downloads the ticket image.

/tickets/<id>/attachments/<attachment_id>/view/ | ticket_attachment_view | Streams an attachment inline where possible.

/tickets/<id>/attachments/<attachment_id>/download/ | ticket_attachment_download | Downloads a ticket chat attachment.

Support URLs

URL | View | Purpose

/tickets/support/ | support_dashboard | Support dashboard with queue counts, workload, and high-level ticket summaries.

/tickets/support/queue/ | support_queue | Main support queue for normal tickets with filters for status, priority, department, branch, requester, assignee, and date range.

/tickets/support/department/ | support_department_tickets | Department-specific ticket view split into unassigned, assigned, and solved groups.

/tickets/support/cbs-access/ | support_cbs_access_requests | Support queue focused on CBS access request tickets.

/tickets/support/incidents/ | support_incident_tickets | Support queue focused on incident tickets.

/tickets/support/users/ | support_users | Shows currently logged-in users and user activity information.

/tickets/agent-workload/ | agent_workload_view | Read-only workload page for agents when enabled in authentication settings.

7. Main Workflows

Normal Ticket Creation

- User opens /tickets/new/.
- create_ticket builds a TicketForm with the current user.
- On valid POST, the view creates a Ticket, calculates priority, stores attachments, assigns a user if an assignment email was supplied, and sends notification email where applicable.
- If the request type is incident, the view prepares an IncidentReport template linked to the ticket.
- The user is redirected to the ticket detail page.

Ticket Detail, Chat, and Attachments

- ticket_detail loads the ticket and checks whether the current user is allowed to view it.
- The detail context includes ticket data, approval data, incident report data, chat state, WebRTC call settings, messages, attachments, and action flags.
- Ticket chat uses WebSockets through TicketChatConsumer. Attachments are uploaded over HTTP by ticket_attachment_upload, then broadcast to the chat group.
- Notifications are sent to browser notification groups using helpers in tickets/notifications.py.

Support Queue and Ticket Update

- Support users open /tickets/support/queue/ or one of the specialized support queues.
- Filter helpers such as _get_support_filters and _apply_support_filters convert query parameters into queryset filters.
- A support user opens the update page for a ticket.
- ticket_update delegates to _ticket_update_protected, which validates permissions and status transition rules.
- When a ticket is resolved, the assigned agent must be the actor. The system records resolution data and emails the requester with a signed close link.
- When a ticket is closed, the system records the closing user and blocks invalid reopening from the normal ticket panel.

Remote Access Request

- User opens `/tickets/remote-access-request/`.
- `remote_access_request` validates `RemoteAccessRequestForm`.
- The view creates a ticket and linked `RemoteAccessApproval`.
- Reviewer users receive notifications or email.
- `remote_access_approval_update` handles reviewer decisions and advances the approval status.

CBS Access Request

- Head Office requests use `/tickets/cbs-access-request/`; branch requests use `/tickets/cbs-access-request/branch/`.
- Both routes flow through `_cbs_access_request_view` with different office type settings.
- The view validates `CBSAccessRequestForm`, creates a ticket, creates or updates `RemoteAccessApproval`, stores signoff chain users, and prepares request descriptions/documents.
- Recommender, second recommender, and approver actions are handled by approval update and signoff-chain views.
- After final approval, support can assign and process the approved CBS request.

Incident Report

- Incident tickets have a linked incident report template.
- `ticket_incident_report` creates or updates the structured incident report.
- Registered user, notified users, incident commander, and configured signoff users can sign or reject depending on role and sequence.
- Download views generate incident report files from the saved report data and attachments.
- Resolution is blocked until required incident report conditions are satisfied.

Technical Documents

- `tech_docs` lists technical documents visible to the current user.
- `tech_docs_upload` allows authorized users to upload PDF or Excel files and configure branch/department visibility.
- `tech_doc_view` renders PDFs inline or previews supported Excel files.
- `tech_doc_download` streams the original file.
- `tech_doc_delete` removes documents for users with delete permission.

8. WebSocket Flow

`helpdesk/helpdesk/asgi.py` routes HTTP traffic to Django and WebSocket traffic to `tickets.routing.websocket_urlpatterns`.

WebSocket URL | Consumer | Purpose

`/ws/tickets/<ticket_id>/` | `TicketChatConsumer` | Real-time ticket chat. Checks authentication, ticket chat access, locked-ticket status, saves messages, broadcasts chat events, and sends notifications.

`/ws/tickets/<ticket_id>/call/` | `TicketCallConsumer` | WebRTC call signaling for ticket participants. Relays ring, ready, offer, answer, ICE, and hangup events.

`/ws/notifications/` | `NotificationsConsumer` | User-specific browser notification channel.

9. Supporting Code Responsibilities

File | What It Does

`tickets/chat_rules.py` | Determines when ticket chat is locked and what message should be shown.

`tickets/notifications.py` | Builds chat and call notification payloads and determines notification target users.

`tickets/storage.py` | Custom storage used for uploaded ticket-related files.

`tickets/minio.py` | S3/MinIO client configuration and connection helpers used by storage and download flows.

`tickets/purge.py` | Deletion helpers used by retention and cleanup workflows.

`tickets/context_processors.py` | Adds ticket-related context data to templates, such as login flash announcements.

`accounts/context_processors.py` | Adds authentication mode flags to templates.

`accounts/utlis.py` | Shared account utilities such as outgoing email selection and session logout helpers.

10. How to Read or Extend This Project

- Start from `helpdesk/helpdesk/urls.py` to find which app receives the request.
- Open the app-level `urls.py` to identify the view name.
- Read that view in `views.py`. The view usually shows the full workflow: permission checks, model queries, form validation, save actions, email/notification actions, and template rendering.
- Open the form class in `forms.py` to understand validation and user-facing fields.
- Open the model class in `models.py` to understand stored data, status values, relationships, and helper properties.
- Open the template referenced by `render()` to understand what the user sees.
- For real-time behavior, read `asgi.py`, `tickets/routing.py`, and `tickets/consumers.py`.